

Masab Zia

24P-3034

Saad Bin Sharif

24P-3077

SE-3A

Project Report

Project Overview:-

This project is a **Loan Calculator application** that can calculate different loan values, such as monthly payment, remaining balance, number of payments, interest rate, and loan amount. It supports both a **Graphical User Interface (GUI)** and **Command Line Interface (CLI)**.

The GUI version lets the user enter loan details and see the results visually, while the command-line mode allows calculations using different command-line arguments.

The project includes:

- A core class called **LoanCalculator** that performs all financial calculations.
- A **Qt-based GUI application** (LoanCalcQtMainWindow) to interact with the user.
- A **command-line mode** that can run different calculations depending on options passed.
- Support for reading loan values from a configuration file (**config.txt**).
- **GoogleTest unit tests** to ensure correctness and stability.
- **Doxygen documentation** to automatically generate HTML documentation for the code.

The purpose of this project is to:

- provide correct loan financial calculations,
- allow flexible ways to run the tool (GUI or CLI),
- test important scenarios like normal EMI, invalid inputs, and large tenures,
- and ensure maintainability through proper documentation.

Code snippets of bug fixes and refactorings

Bug 1:

You can currently pass:

- negative loan amount
- negative interest
- negative periods
- zero/negative payment

The calculator will then:

- divide by zero
- take log of negative number
- produce undesired results

Fix

Add validation inside setter functions.

```
inline void setAmount(float A) {  
    if(A <= 0)  
        throw std::invalid_argument("Loan amount must be > 0");  
    amount_ = A;  
    amountSet_ = true;  
}
```

```
inline void setInterest(float i) {  
    if(i <= 0)  
        throw std::invalid_argument("Interest must be > 0");  
    interest_ = i;  
    interestPeriodic_ = i/100.0/12.0;  
    interestSet_ = true;  
}
```

```
inline void setPeriodTotal(int N) {  
    if(N <= 0)  
        throw std::invalid_argument("Total period must be > 0 months");  
    periodTotal_ = N;  
    periodTotalSet_ = true;  
}
```

Bug 2:

If:

`periodTotal_ = 360 (30 years)`

then $(1+i)^{360}$ can overflow `float` precision and return garbage.

Cause

Everything is calculated in `float`.

Fix

Use safer precision (`long double`) everywhere for core formulas.

```
long double i = (long double)interestPeriodic_;
```

```
long double totalAmount = (long double)amount_ - initialPayment_;
```

```
totalAmount = totalAmount + openingFee_ + (totalAmount * (openingPercent_/100.0));
```

```
long double denom = (1 - pow((1+i), (-1*(long double)periodTotal_)));
```

```
long double result = (i * totalAmount) / denom;
```

```
return (float)result;
```

Bug 3:

Right now:

opening fee, opening %, interest, etc default values are hardcoded values in code.

Requirement:

Removing hardcoded values by reading configuration from a file (`config.txt`)

Solution:

create a config file

```

// load config
{
    ifstream f("config.txt");
    if(f.good()){
        string key;
        long double val;
        while(f>>key>>val){
            if(key=="default_interest")
                calculator.setInterest(val);
            else if(key=="default_opening_fee")
                calculator.setOpeningFee(val);
            else if(key=="default_opening_percent")
                calculator.setOpeningPercent(val);
        }
    }
}

```

Bug 4:

The formula $N = -\log(1 - i \cdot A/P) / \log(1+i)$ is only valid if $1 - i \cdot A/P > 0$. If the monthly `payment_` is too small ($\leq i \cdot \text{amount_}$) then the expression inside `log( )` is ≤ 0 , causing domain error (crash).

```

float LoanCalculator::calculateNumberPayments()
{
    if(!amountSet_ || !interestSet_ || !paymentSet_)
    {
        throw invalid_argument("Must set loan amount, interest, and payment for this calculation" );
    }
}

```

```

}

long double i = (long double)interestPeriodic_;
long double A = (long double)amount_;
long double P = (long double)payment_;

// argument to log: 1 - i*A/P
long double arg = 1.0L - (i * A / P);
if(!(arg > 0.0L)) // handles nan/inf cases too
{
    throw invalid_argument("Payment is too small (<= interest * amount). The loan will never
be repaid with the current payment.");
}

long double numerator = -1.0L * log10l(arg);
long double denominator = log10l(1.0L + i);

if(!(denominator > 0.0L))
{
    throw invalid_argument("Invalid periodic interest value (non-positive).");
}

long double result = numerator / denominator;
return (float)result;
}

```

Refactorings:-

LoanCalculator.cpp file

```
double LoanCalculator::calculateLoanB()
{
    if(!amountSet_ || !interestSet_ || !periodElapsedSet_ || !paymentSet_)
        throw std::invalid_argument("Must set loan amount, interest, and elapsed
period for this calculation");

    if(interestPeriodic_ == 0)
        throw std::invalid_argument("");

    double factor = std::pow(1 + interestPeriodic_, periodElapsed_);

    return (amount_ * factor) -
        ((payment_ / interestPeriodic_) * (factor - 1));
}

/*
 * Payment amount:
 *  $P = i \cdot A / (1 - (1+i)^{-N})$ 
 */
double LoanCalculator::calculatePayme()
{
    if(!amountSet_ || !interestSet_ || !periodTotalSet_)
        throw std::invalid_argument("Must set loan amount, interest, and total
period");

    if(interestPeriodic_ == 0)
        throw std::invalid_argument("");

    double totalAmount = amount_ - initialPayment_;
    totalAmount += openingFee_ + (totalAmount * (openingPercent_ / 100.0));

    return (interestPeriodic_ * totalAmount) /
        (1 - std::pow(1 + interestPeriodic_, -periodTotal_));
}

/*
 * Number of payments:
 *  $N = -\ln(1 - i \cdot A / P) / \ln(1+i)$ 
 */
double LoanCalculator::calculateNumbe()
{
    if(!amountSet_ || !interestSet_ || !paymentSet_)
```

```

        throw std::invalid_argument("Must set loan amount, interest, and payment");

    if(interestPeriodic_ == 0)
        throw std::invalid_argument("");

    double ratio = 1.0 - (interestPeriodic_ * amount_ / payment_);

    if(ratio <= 0)
        throw std::invalid_argument("Invalid values: loan or payment too small");

    return -std::log(ratio) / std::log(1.0 + interestPeriodic_);
}

/*
 * Original loan amount:
 *  $A = (P/i) * (1 - (1+i)^{-N})$ 
 */
double LoanCalculator::calculateLoanA()
{
    if(!paymentSet_ || !interestSet_ || !periodTotalSet_)
        throw std::invalid_argument("Must set payment, interest, and total
period");

    if(interestPeriodic_ == 0)
        throw std::invalid_argument("");

    return (payment_ / interestPeriodic_) *
        (1 - std::pow(1 + interestPeriodic_, -periodTotal_));
}

/*
 * Approximate interest rate:
 *  $i = (((1 + P/A)^{(1/q)} - 1)^q - 1)$ 
 * where  $q = \ln(1+1/N) / \ln(2)$ 
 */
double LoanCalculator::calculateInter()
{
    if(!amountSet_ || !paymentSet_ || !periodTotalSet_)
        throw std::invalid_argument("Must set amount, payment, and total period");

    double q = std::log(1.0 + 1.0 / periodTotal_) / std::log(2.0);
    double base = std::pow((1.0 + payment_ / amount_), 1.0 / q) - 1.0;
    double monthlyInterest = std::pow(base, q) - 1.0;

    return monthlyInterest * 12 * 100; // annual percentage
}

```



```

double LoanCalculator::calculateEffec()
{
    if(!amountSet_ || !periodTotalSet_)
        throw std::invalid_argument("Must set amount and total period");

    double payment = calculatePayment();
    double totalAmount = amount_ - initialPayment_;

    double q = std::log(1.0 + 1.0 / periodTotal_) / std::log(2.0);
    double base = std::pow((1.0 + payment / totalAmount), 1.0 / q) - 1.0;
    double monthlyInterest = std::pow(base, q) - 1.0;

    return monthlyInterest * 12 * 100;
}

```

loanCalculator.h file

```

inline void setAmount(double A) { amount_ = A; amountSet_ = true; }
inline double getAmount() const { return amount_; }

// Initial down payment
inline void setInitialPayment(double initialA) { initialPayment_ =
initialA; }
inline double getInitialPayment() const { return initialPayment_; }

/*
Yearly interest rate (e.g. 6.75)
Internally stored as periodic (monthly) rate: i/100/12
*/
inline void setInterest(double i)
{
    interest_ = i;
    interestPeriodic_ = i / 100.0 / 12.0;
    interestSet_ = true;
}

inline double getInterest() const { return interest_; }
inline double getPeriodicInterest() const { return interestPeriodic_; }

// Monthly payment
inline void setPayment(double P) { payment_ = P; paymentSet_ = true; }
inline double getPayment() const { return payment_; }

// Total loan period (N)
inline void setPeriodTotal(int N) { periodTotal_ = N; periodTotalSet_ =
true; }

```

```

inline int  getPeriodTotal() const    { return periodTotal_; }

// Elapsed payment periods (n)
inline void setPeriodElapsed(int n)  { periodElapsed_ = n; periodElapsedSet_
= true; }
inline int  getPeriodElapsed() const { return periodElapsed_; }

// Optional opening fees
inline void setOpeningFee(double fee)    { openingFee_ = fee; }
inline double getOpeningFee() const      { return openingFee_; }

inline void setOpeningPercent(double percent) { openingPercent_ = percent; }
inline double getOpeningPercent() const      { return openingPercent_; }

// Reset all variables and flags
inline void reset()
{
    amount_ = initialPayment_ = interest_ = interestPeriodic_ =
    payment_ = openingFee_ = openingPercent_ = 0.0;

    periodTotal_ = periodElapsed_ = 0;

    amountSet_ = interestSet_ = paymentSet_ =
    periodTotalSet_ = periodElapsedSet_ = false;
}

//
// Calculation Methods
//

double calculateLoanBalance();
double calculatePayment();
double calculateNumberPayments();
double calculateLoanAmount();
double calculateInterestRate();
double calculateEffectiveInterestRate

std::string toString();

private:
    double amount_;           // principal A
    bool amountSet_;

    double initialPayment_;   // initial down payment

    double interest_;         // yearly interest %
    double interestPeriodic_; // monthly interest rate (decimal form)

```

```

bool interestSet_;

double payment_;           // payment P
bool paymentSet_;

int periodTotal_;          // N
bool periodTotalSet_;

int periodElapsed_;        // n
bool periodElapsedSet_;

double openingFee_;        // flat fee
double openingPercent_;    // % fee
};

```

Overall many refactorings is done in the code including all files (loanCalculator.h, LoanCalculatorMain.cpp)

Test Output:-

```

● saad@fedora:~/Documents/Semester-3/legacy-calc-2009/src$ ./test_emi
Running main() from /builddir/build/BUILD/gtest-1.15.2-build/googletest-1.15.2/googletest/src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanCalculatorTest
[ RUN      ] LoanCalculatorTest.NormalEmiCalculation
[      OK  ] LoanCalculatorTest.NormalEmiCalculation (0 ms)
[ RUN      ] LoanCalculatorTest.InvalidInputHandling
[      OK  ] LoanCalculatorTest.InvalidInputHandling (0 ms)
[ RUN      ] LoanCalculatorTest.LargeTenureNoOverflow
[      OK  ] LoanCalculatorTest.LargeTenureNoOverflow (0 ms)
[-----] 3 tests from LoanCalculatorTest (0 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (0 ms total)
[ PASSED  ] 3 tests.
○ saad@fedora:~/Documents/Semester-3/legacy-calc-2009/src$

```

Git commit history:-

Commits

dev

All users

All time

Commits on Nov 25, 2025

final refactoring and doxygen html

muhammadsaad0077 committed 1 minute ago

e3a6c1e

<>

Commits on Nov 24, 2025

test cases_2, doxygen commenting and file

muhammadsaad0077 committed 18 hours ago

d46cf5c

<>

test_cases

muhammadsaad0077 committed 18 hours ago

5b39f8c

<>

bug fix_2

muhammadsaad0077 committed 19 hours ago

c8a8f5d

<>

fixing bugs

muhammadsaad0077 committed 2 days ago

03e8a4c

<>

Commits on Nov 23, 2025

intial setup

muhammadsaad0077 committed 2 days ago

d66e62e

<>

Commits

dev

All users

All time

Commits on Nov 25, 2025

final changes

Masaab-Zia634 committed 2 minutes ago

c0b3b0f

<>

doxygen commenting

Masaab-Zia634 committed 1 hour ago

cd1e13e

<>

Commits on Nov 24, 2025

optimization:header and main cpp file

Masaab-Zia634 committed 18 hours ago

772fca7

<>

refactoring: mainCpp file optimization

Masaab-Zia634 committed 19 hours ago

6f29b29

<>

Commits on Nov 23, 2025

project setup

Masaab-Zia634 committed 2 days ago

b3ef083

<>

Link: <https://github.com/Masaab-Zia634/legacy-calc-2009/tree/dev>

Link: <https://github.com/muhammadsaad0077/legacy-calc-2009/tree/dev>